

iLabAfrica Research Centre
Strathmore University
AI Practitioner Professional Programme (AIPP)



SureRide

AI-Powered Ride Booking Agent

An end-to-end conversational AI system to eliminate drunk-driving fatalities in Kenya through frictionless, safe ride booking.

Capstone Executive Portfolio Report

Academic Year 2026 — June Session

CONFIDENTIAL — ACADEMIC SUBMISSION

Table of Contents

Table of Contents	2
0. Executive Summary	3
Key System Metrics	3
Projected Business & Technical Outcomes	3
1. Problem Framing	4
1.1 The Problem - Drunk Driving in Kenya	4
1.2 Why an AI Agent is the Right Solution	4
1.3 Business Context	4
2. System Architecture & Technical Execution	5
2.1 Architecture Overview	5
2.2 Booking Conversation Flow	5
2.3 State Schema	5
2.4 Pricing Engine	5
2.5 Test Coverage	6
2.6 Tech Stack	6
3. Prompt Engineering Strategy	7
3.1 RAG FAQ Grounding Prompt	7
3.2 Deterministic Node Strategy (Anti-Prompt-Injection)	7
3.3 Context Optimisation	7
4. AI Ethics & Governance	8
4.1 Governance Structure	8
4.2 KDPA 2019 Compliance	8
4.3 Risk Register	8
4.4 Explainability	9
4.5 Impaired-User Safety Considerations	9
4.6 Responsible AI Design Decisions	9
5. Integration of Learning	10
5.1 Three Curriculum Pillars — Technical Module Targets	10
6. Reflection & Architectural Limitations	11
6.1 What Worked Well	11
6.2 Technical Challenges Encountered	11
6.3 Current Limitations	11
6.4 Recommended Next Steps	12
6.5 Lessons Learned	12

0. Executive Summary

SureRide is an AI-powered conversational agent designed to address one of Kenya's most acute road-safety challenges: drunk driving. According to the National Transport and Safety Authority (NTSA), alcohol-related crashes account for over 30% of annual road fatalities in Kenya. The core friction is not a lack of awareness rather it is the difficulty of booking a safe ride home late at night, when impairment degrades decision-making and digital fluency.

SureRide eliminates this friction by deploying a stateful LangGraph conversational agent backed by a Retrieval-Augmented Generation (RAG) knowledge base and a live PesaPal payment gateway. The system guides a user from greeting to confirmed booking in under 60 seconds, via a familiar WhatsApp-style chat interface, without requiring any app download or account creation.

Key System Metrics

Agent States	Pilot Zones	AIPP Modules	Risks Catalogued
10	19	6	8

Projected Business & Technical Outcomes

Metric	Baseline / Benchmark	SureRide Target	Measurement Method
Booking completion rate	~40% for typical 5-step ride-hailing funnels (industry benchmark)	≥70% — single-answer-per-turn UX reduces drop-off	Streamlit session logs: done states ÷ greet states
Time to confirmed booking	Uber/Bolt: ~90–120 s (account required, multi-screen)	≤60 s end-to-end (greet → payment URL issued)	Timestamp delta: greet entry → await_payment entry
FAQ hallucination rate	Free-form LLM: est. 15–25% factual error on pricing/policy queries	~0% — RAG-grounded, source-constrained prompt	Manual audit of 50 sampled FAQ responses vs sureride_faq.txt
Pilot addressable incidents	NTSA: >30% of ~3,000 annual Nairobi road fatalities are alcohol-related (~900+)	~2,400 at-risk journeys/month captured across 19 pilot zones at peak hours	Zone demand model: est. avg 4 bookings/zone/peak night × 19 zones × 32 peak nights/month
Unit economics at scale	Average Westlands → Karen fare: KES 695 (12 km benchmark)	KES 50 net booking fee per completed ride; breakeven at ~600 rides/month	Pricing engine: FareBreakdown.booking_fee as platform revenue line
Test coverage	—	13 automated pytest cases — 100% pass rate on pricing and zone logic	pytest tests/ -v (included in submission)

1. Problem Framing

1.1 The Problem - Drunk Driving in Kenya

Drunk driving is a preventable public health crisis. The NTSA attributes more than 30% of Kenya's annual road accident fatalities to alcohol-impaired driving. A figure that spikes significantly on Friday and Saturday nights. The problem is not purely behavioural: impaired individuals intend to get home safely but lack a fast, low-friction way to do so at 2 AM outside a Nairobi nightspot.

Existing ride-hailing applications (Uber, Bolt, Little) require account registration, credit card setup, and reliable mobile data and thus barriers that are disproportionately high for impaired users. The friction of multi-step app navigation in a degraded cognitive state is precisely the gap SureRide is designed to close.

1.2 Why an AI Agent is the Right Solution

AI Justification: A conversational agent mimics WhatsApp which is the most familiar digital interface for Kenyan mobile users. It guides the user through a booking with natural-language prompts, requiring only one answer at a time, dramatically lowering cognitive load compared to a traditional form-based app.

Three criteria confirm AI agents as the appropriate solution type:

- **Impact:** Directly addresses a high-severity organisational and societal friction point with a quantifiable target outcome (reduce drunk-driving incidents).
- **Justification:** The use of a stateful LangGraph agent enables multi-turn conversation with memory, something rule-based IVR or simple chatbots cannot achieve reliably.
- **Feasibility:** The project scope (19 pilot zones, single-session booking, simulated driver dispatch) is calibrated for a 12-week design cycle with one developer.

1.3 Business Context

Factor	Detail
Target Users	Adult drivers in Nairobi's affluent zones who need a safe ride home after socialising
Pilot Geography	19 geofenced Nairobi neighbourhoods (CBD, Westlands, Karen, Kilimani, Runda, and 14 others)
Revenue Model	Distance-based fares (KES 150 base + KES 35/km + KES 50 booking fee); 1.5× surge Fri/Sat 10 PM–3 AM
Payment	PesaPal v3 — M-Pesa and card, fully delegated to avoid PCI scope
Regulatory Context	Kenya Data Protection Act (KDPA) 2019; Kenya AI Policy 2024

2. System Architecture & Technical Execution

2.1 Architecture Overview

SureRide is built on four decoupled layers, each independently testable:

Layer 1 — Conversation State Machine (*langgraph_agent*)

A LangGraph StateGraph with a single dispatcher node that reads `current_step` from `BookingState` and delegates to the correct handler. The dispatcher pattern prevents loop traps, makes transitions fully deterministic, and isolates user input from the LLM system prompt.

Layer 2 — RAG Knowledge Base (*rag/*)

`ingest.py` chunks the `sureride_faq.txt` (chunk size 400, overlap 60) and embeds it using Google Gemini embedding-001 into a local ChromaDB vector store. `retriever.py` returns a LangChain retriever (`k=3`) used by the `rag_tool_node` for mid-booking FAQ interception.

Layer 3 — Business Logic Utilities (*utils/*)

`zones.py` — 19 zone centroids with alias lookup, substring match, and difflib fuzzy matching.

`pricing.py` — FareBreakdown dataclass with Haversine × 1.3 road distance and surge logic.

`pesapal.py` — PesaPal v3 OAuth2 flow: token → IPN registration → order submission → status polling.

Layer 4 — Streamlit UI (*ui/app.py*)

WhatsApp-style chat rendered inside an `iframe` component to isolate CSS. Sidebar navigation exposes an Operations Dashboard with KPI metrics, zone demand bar chart, ethics KPIs, and a transaction feed.

2.2 Booking Conversation Flow

The agent moves the user through a 10-step deterministic flow:

greet → collect_name → collect_pickup → validate_pickup → collect_destination → validate_destination → show_fare → confirm_fare → await_payment → done

At any `collect_*` step, if the user's message begins with a question word or ends with '?', the dispatcher intercepts it and routes to `rag_tool_node`, returning an FAQ answer before resuming the booking at the same step.

2.3 State Schema

```
class BookingState(TypedDict):
    messages: Annotated[list, add_messages] # append-only
    user_name, pickup_raw, pickup_zone, destination_raw, destination_zone
    distance_km, fare_kes, surge_applied
    booking_confirmed, awaiting_confirmation
    pesapal_order_id, pesapal_pay_url, payment_status
    current_step: str # flow control
    faq_query, faq_answer # RAG intercept
```

2.4 Pricing Engine

Component	Value
Base fare	KES 150
Per-km rate	KES 35 (on Haversine × 1.3 road distance)
Booking fee	KES 50 (always shown as a separate line item)
Minimum fare	KES 300 total
Surge multiplier	×1.5 on base + distance (not booking fee) — Friday & Saturday 10 PM–3 AM, including Sunday 12–3 AM

2.5 Test Coverage

Two pytest modules provide automated validation of core business logic:

- **tests/test_pricing.py** — 7 surge tests + 6 fare formula tests covering minimum fare, surge activation, surge deactivation, breakdown field integrity, and the Westlands→Karen benchmark range.
- **tests/test_zones.py** — Exact match, case-insensitive, substring, alias, out-of-zone rejection, fuzzy match, distance formula, and coordinate range validation across all 19 zones.

2.6 Tech Stack

Component	Technology	Role
Agent Orchestration	LangGraph 0.1+ / LangChain 0.2+	Stateful multi-turn conversation
Language Model	Google Gemini 1.5 Flash	FAQ synthesis (temperature 0.3)
Embeddings	Gemini embedding-001	RAG document vectorisation
Vector Store	ChromaDB 0.5+	Persisted FAQ knowledge base
UI Framework	Streamlit 1.35+	Chat UI + Operations Dashboard
Payment Gateway	PesaPal v3 REST API	M-Pesa / card checkout
Testing	pytest	Pricing and zone unit tests

3. Prompt Engineering Strategy

3.1 RAG FAQ Grounding Prompt

The primary LLM call in the system is the FAQ synthesis prompt used by `rag_tool_node`:

```
You are the SureRide AI assistant. Use ONLY the context below to answer
the customer's question concisely and in a friendly, conversational
tone.
Do not make up information. Keep the answer under 4 sentences.

Context:
{retrieved_chunks}

Customer question: {user_query}
Answer:
```

Design principles applied:

- **Role constraint** — Sets persona, tone, and domain boundary.
- **Source grounding** — "Use ONLY the context below" prevents hallucinated pricing, ETAs, or phone numbers.
- **Conciseness constraint** — "Under 4 sentences" keeps responses within a WhatsApp bubble.
- **Zero-shot instruction** — "Friendly, conversational tone" ensures natural output without few-shot examples.

3.2 Deterministic Node Strategy (Anti-Prompt-Injection)

Security design decision: For all structured booking steps (name, pickup, destination, confirmation), user input is processed by deterministic Python heuristics — never by an LLM. This renders prompt injection attempts completely ineffective because user messages never reach the LLM system prompt in those steps.

Example from `collect_name_node`:

```
last = state["messages"][-1].content.strip()
name = last.split()[0].capitalize() # first word only, capitalised
```

3.3 Context Optimisation

Parameter	Value	Rationale
Model	Gemini 1.5 Flash	Low latency, 1M token context window, cost-efficient
Temperature	0.3	Low randomness — keeps FAQ answers factual and consistent
Retrieval k	3 chunks	Sufficient context without inflating prompt size
Chunk size	400 chars, 60 overlap	Preserves FAQ Q&A structure; minimises token cost
Embedding model	gemini-embedding-001	Multilingual-capable; same provider as LLM (no cross-API latency)

4. AI Ethics & Governance

4.1 Governance Structure

Role	Responsibility
AI System Owner	Defines purpose, scope, and ethical boundaries of the agent
Data Protection Officer (DPO)	Ensures KDPa compliance; manages data subject access requests
Technical Lead	Maintains LangGraph agent, RAG pipeline, and deployment
Human Oversight Agent	Reviews flagged conversations; authorises refunds and dispute resolutions

4.2 KDPa 2019 Compliance

SureRide applies the Kenya Data Protection Act 2019 principles across the full data lifecycle:

Data Collected	Purpose	Retention
First name	Personalise conversation	Session only — never persisted to database
Pickup location	Dispatch driver	30 days, then deleted
Destination	Route planning	30 days, then deleted
Booking timestamp	Audit trail	12 months
PesaPal Order ID	Payment status validation	12 months (accounting / audit)

Not collected: Phone number, national ID, full home address, raw card numbers, mobile money PINs, or device identifiers. All payment transactions are delegated to PesaPal's PCI-DSS compliant hosted pages.

4.3 Risk Register

#	Risk	Level	Primary Mitigation
R 1	Hallucination in booking details	 High	Human-in-the-loop confirmation + RAG grounding
R 2	Location data exposure	 Med	Session-only state; no database; PII never sent to LLM
R 3	Algorithmic bias in driver matching	 Med	No ML matching in prototype; bias audit required pre-production
R 4	Prompt injection	 Med	Dispatcher pattern: user input never in LLM system prompt
R 5	Gemini API unavailability	 High	try/except on all external calls; safe fallback messages
R 6	Passenger misidentification / impaired-user safety	 Med	Name + fare summary shown pre-confirmation; driver briefed to verify name verbally; human escalation path active ('help' → live agent)
R 7	Sensitive disclosure in chat	 Med	Human escalation path: type 'help' → live agent

R 8	Payment failure / fraud	 High	Delegated to PesaPal; OAuth2 token validation; manual polling fallback
--------	-------------------------	---	--

4.4 Explainability

The SureRide agent is fully deterministic and rule-based at the conversation-flow level. Every decision traces back to a single field in `BookingState.current_step`. An auditor can inspect the full state at any point to understand exactly what the agent recorded and why. There is no black-box decision process in the booking pipeline.

For RAG answers, the retrieved source chunks are traceable to specific lines in `sureide_faq.txt`, making every FAQ answer independently verifiable against the company knowledge base.

4.5 Impaired-User Safety Considerations

SureRide's primary user is, by definition, cognitively impaired at the point of booking. This creates a safety obligation that goes beyond standard AI ethics frameworks and warrants explicit design treatment:

Identity verification gap: The current prototype accepts any first name supplied by the user with no phone number, OTP, or account check. A sober third party could theoretically book a ride on behalf of an impaired person without their knowledge. Pre-production deployment must address this via a lightweight OTP sent to the user's M-Pesa number — already captured implicitly during PesaPal checkout — which closes the loop without requiring account creation.

- **One-question-at-a-time UX:** The dispatcher pattern is itself a safety design — it never presents multiple inputs simultaneously, matching the reduced working-memory capacity of an impaired user.
- **No cancellation penalty:** The current fare model does not charge cancellation fees, intentionally removing a barrier that might discourage a user from abandoning an unsafe situation.
- **Human escalation always available:** Typing 'help' at any booking step routes to a live agent message, ensuring the system degrades safely when the AI cannot handle a distressed or confused user.
- **Pre-production requirement:** A formal usability test with participants simulating mild impairment must be conducted before public launch. Completion rate and error rate under cognitive load are the key metrics.

4.6 Responsible AI Design Decisions

Decision	Rationale
No persistent user database	Eliminates the largest KDPA risk — no database means no breach surface
Hard-coded conversation flow	Prevents LLM from making promises SureRide cannot fulfil (e.g., invented pricing)
Explicit human-in-the-loop confirmation	Industry best practice for consequential actions; reduces hallucination impact
RAG over pure LLM generation	Grounds answers in verified policy — prevents model from inventing prices or policies
Fallback messages on all external calls	Graceful degradation: users receive honest, safe responses even during API outages

Open-source core infrastructure	LangGraph, ChromaDB, Streamlit are auditable; no opaque vendor lock-in
---------------------------------	--

5. Integration of Learning

SureRide integrates all six AIPP curriculum modules into a single, coherent production-ready system. The cross-disciplinary requirement — that Prompt Design and Ethical Governance must be visible in the code design — is met explicitly throughout the architecture.

AIPP Module	SureRide Component	Where in Code
Module 1 — AI Foundations	Problem framing using AI theory: defining the correct agent type (conversational), identifying hallucination risk, selecting RAG over pure LLM	Architecture decisions; governance_ethics.md §4
Module 2 — Prompt Engineering	RAG synthesis prompt with role constraint, source grounding, conciseness limit; deterministic node strategy as anti-injection design	langgraph_agent/nodes.py:rag_tool_node; docs/prompt_logs.md
Module 3 — AI Agents & Orchestration	LangGraph StateGraph with dispatcher pattern; 10-step conversation FSM; conditional FAQ routing; multi-turn state management	langgraph_agent/agent.py, nodes.py, state.py
Module 4 — Retrieval-Augmented Generation	ChromaDB vector store; Gemini embedding-001; document chunking strategy (400/60); top-k retrieval; grounded synthesis	rag/ingest.py, rag/retriever.py
Module 5 — AI Ethics & Governance	KDPA compliance framework; 8-risk register; mitigation matrix; explainability design; responsible AI decisions	docs/governance_ethics.md; risk mitigations in all nodes
Module 6 — AI Applications	End-to-end Streamlit deployment; WhatsApp-style UI; Operations Dashboard with KPI metrics; PesaPal live payment integration	ui/app.py; utils/pesapal.py

5.1 Three Curriculum Pillars — Technical Module Targets

The evaluation framework requires weaving logic across three foundational areas:

Deep System Models & Intelligence

Gemini 1.5 Flash as the LLM backbone; Gemini embedding-001 for dense vector representation; cosine similarity search in ChromaDB; Haversine formula for geospatial distance.

Functional Backend Computation Engines

PesaPal v3 OAuth2 REST integration; FareBreakdown pricing dataclass; fuzzy zone matching with difflib; pytest unit test suite with 13 test cases.

Multi-Agent Workflow Automation Models

LangGraph dispatcher pattern with 10 specialist nodes; conditional routing (FAQ intercept, auto-advance steps); state accumulation across turns; graceful degradation on all external service calls.

6. Reflection & Architectural Limitations

6.1 What Worked Well

- **Dispatcher pattern in LangGraph** — The single-node, step-routing architecture proved highly robust. It eliminated loop traps that appeared in early multi-node experiments and made debugging trivial: every failure traced to a single handler function.
- **RAG grounding** — Restricting the LLM to 'ONLY the context below' completely eliminated hallucinated prices and policies in FAQ answers. Every answer was traceable to a specific line in `sureide_faq.txt`.
- **PesaPal integration** — Implementing a real payment gateway (with mock fallback) rather than a simulated one gave the demo genuine credibility. The mock mode transparently labels itself, satisfying both demo practicality and ethical honesty requirements.
- **WhatsApp-style UI** — Rendering chat bubbles inside an iframe isolated the CSS from Streamlit's stylesheet, producing a visually authentic interface that resonated strongly with the target user context.

6.2 Technical Challenges Encountered

- **Streamlit iframe rendering** — Streamlit's layout engine strips custom CSS from `st.markdown()`. Solution: use `components.html()` to render an isolated iframe with full CSS control, at the cost of added height-estimation complexity.
- **PesaPal webhook limitations** — The local Streamlit server has no public callback URL, making PesaPal's IPN webhook system unusable in development. Mitigation: implemented a manual polling button that queries the PesaPal status API directly using OAuth2 token validation.
- **LangGraph state accumulation** — The `add_messages` reducer appends messages rather than replacing them, causing early versions to re-trigger greet after every turn. Fixed by adding the `'not messages or messages[-1].type != "human"'` guard in the dispatcher.

6.3 Current Limitations

Limitation	Impact
No real driver dispatch	Booking confirmation is a simulation; no actual driver is assigned
Single-session memory	Returning users start fresh — no booking history or preferences
Heuristic name parsing	First-word split fails for compound names or greeting responses ('Hi, I'm John')

Haversine distance only	Road distance = haversine × 1.3 (fixed tortuosity factor); no real routing data
English only	No Swahili support — excludes a significant portion of Kenyan users
19 zones only	Peri-urban areas (Thika, Ruiru, Athi River) are excluded from pilot

6.4 Recommended Next Steps

1. Integrate Google Maps Directions API for exact road distance (replace Haversine × 1.3).
2. Add Swahili language support using a multilingual LLM (Gemini 1.5 Pro natively supports Swahili).
3. Replace heuristic name extraction with a named-entity recognition (NER) model.
4. Implement KDPA-compliant PostgreSQL persistence for booking history with row-level encryption.
5. Conduct a formal algorithmic bias audit before any production driver-matching ML model.
6. Commission an independent KDPA Data Protection Impact Assessment (DPIA).
7. Deploy on a public cloud endpoint (Cloud Run / Railway) with a real PesaPal IPN webhook URL.

6.5 Lessons Learned

The most valuable lesson from this capstone was that **responsible AI design is not an afterthought**. The choice to use a hard-coded LangGraph dispatcher rather than a free-form LLM was driven equally by ethical reasoning (preventing hallucinated bookings) and engineering preference. The decision to delegate payment to PesaPal's hosted pages was the correct KDPA compliance call, even though it added integration complexity. This principle that safety constraints and ethical guardrails often produce better engineering outcomes is one to carry into all future AI system design.